

Remembered, Not Generated: A Renamed-Schema Control for LLM-Provisioned Test Data, the Cost of Planning Instead of Emitting, and What Valid Data Does Not Do for LLM-Generated Tests

Vijay Prasad Javvadi*

Independent Researcher, Plainsboro, NJ, USA

Abstract

Large language models are increasingly asked to supply test data for relational schemas. We report three executable experiments. First, we measure how close LLM-generated data comes to the fixtures that five open-source PostgreSQL schemas ship, then repeat the measurement after renaming every table and column to a synonym. Directly emitted rows match the human fixtures on the original schemas (median Jensen–Shannon distance 0.25 over the ten paired cells; 0.00 for Claude on the three widely mirrored schemas) and lose the match when the identifiers change (0.73; $p = 0.016$), while their validity is unchanged ($p = 1.0$) and run-to-run variation is at most 0.06: the apparent realism is recall. Second, we compare direct emission with a plan-then-execute design in which the model writes a generation plan that a constraint-aware engine executes. Neither is more valid at 500 rows per table (medians 100 % and 98.7 %), but the engine never violates a foreign key, CHECK or NOT NULL constraint, whereas emitted rows violate foreign keys and uniqueness on every schema that declares them; planning uses 29.5 times fewer tokens. Third, we inject the generated fixtures into an existing LLM test-generation pipeline with 293 frozen mutants. No fixture reliably changes the kill rate. Provisioned rows lower the share of usable generated tests, because the tests assume an empty database; a short statement of the database state in the prompt restores the yield (93 %) without changing the

*Corresponding author. ORCID: 0009-0004-1192-6906.

Email address: vijay@vijayjavvadiresearch.ai (Vijay Prasad Javvadi)

kill rate. Every input is checksummed and every number is regenerated by released scripts.

Keywords: test data generation, large language models, memorisation, benchmark contamination, referential integrity, mutation testing, empirical software engineering

1. Introduction

A test needs data. A generated test for a service that lists orders exercises nothing unless orders exist; a fixture whose foreign keys point nowhere does not load. Developers now routinely ask a large language model (LLM) to produce test data for a schema, and the model usually obliges with rows that look plausible. This paper asks three questions about that practice and answers each with an executable experiment.

The first question is whether the plausibility is real. The schemas on which such tools are demonstrated and evaluated—Sakila, Northwind, Chinook—have been mirrored in tutorials and repositories for two decades, together with their sample data. A model that reproduces the sample data has not generated anything. We measure the fidelity of LLM-emitted rows to the shipped human fixtures of five open-source schemas, then rename every table and column to a synonym, remove the dataset names, and measure again. The difference is what recall contributed.

The second question is what a different mechanism buys. Instead of emitting rows, the model can write a compact generation plan (which generator to use per column, which distributions, which references) that a deterministic engine executes in dependency order, so that keys, uniqueness and CHECK constraints are the engine’s responsibility rather than the model’s. Synth-Data [1], the tool studied here, implements this design. We compare it with the most favourable direct-emission harness we could build, on the same schemas, scoring every emitted row by inserting it into PostgreSQL with all constraints enabled.

The third question is whether any of this matters downstream. We take an LLM test-generation pipeline with an executable mutation benchmark developed in our earlier work (manuscript under review) [2], pre-load its database-backed services with the generated fixtures, describe those fixtures to the generator, and measure the share of usable tests and the mutation kill

30 rate against the pipeline’s own baseline. The hypothesis stated in advance
31 was that constraint-valid data would improve both.

32 *Findings..* Direct emission’s fidelity to the human fixtures is largely memo-
33 risation: the categorical distance to the human data rises from a median of
34 0.25 to 0.73 when identifiers are renamed, far above the run-to-run variation
35 of at most 0.06, while validity is unchanged. Plan-then-execute and direct
36 emission reach the same validity at 500 rows per table, but for different rea-
37 sons and at a thirtieth of the token cost; on declared-domain columns both
38 arms match the human distributions, and the fidelity gap lies entirely in
39 free-text columns, which is where recall operates. Downstream, provisioned
40 fixtures do not raise the kill rate and reduce usable-test yield, because gen-
41 erated tests are written for an empty database. The advance hypothesis is
42 rejected, the mechanism is identified, and a follow-up arm shows that stating
43 the database state explicitly in the prompt restores the yield while leaving
44 the kill rate where it was.

45 *Contributions..* (i) A renamed-schema control for LLM-generated relational
46 data, with the first measurement of how much of the fidelity reported on
47 public schemas is recall (Section 4). (ii) An executable benchmark of LLM
48 test-data provisioning against five human fixture sets, with a PostgreSQL
49 scorer that classifies every rejected row, and the finding that validity comes
50 from the engine, not the model (Section 5). (iii) A downstream experiment
51 showing that constraint-valid fixtures neither help nor reliably hurt an LLM
52 test generator’s kill rate and lower its usable yield, the fixture-contract mech-
53 anism behind the loss, and a follow-up arm in which an explicit database-
54 state contract removes the loss (Section 6). (iv) A defect trail for the studied
55 engine: five defects fixed before the study version was frozen and four found
56 by LLM-written plans afterwards, reported as results (Section 7).

57 **2. Background**

58 *2.1. Direct emission*

59 The model receives the DDL, a description of the business domain and a
60 row target, and answers with rows. A single response cannot hold thousands
61 of rows, so the harness requests rows in chunks, table by table in foreign-
62 key order, and shows the model a sample of the parent keys it has already
63 emitted so that references can be valid. This is the naive approach in its

64 most favourable form: the model is told everything it needs to satisfy every
65 constraint. Cost scales with the number of rows, and every row’s constraints
66 are the model’s to satisfy.

67 2.2. Plan-then-execute

68 SynthData¹ parses the DDL into tables, columns, types, primary and
69 foreign keys, UNIQUE and CHECK constraints (including IN lists and enum
70 types). It asks the model once for a YAML plan: one generator per column
71 from a fixed vocabulary (`sequence`, `fk`, `choice` with weights, `int`, `float` and
72 `lognormal` ranges, `date` and `datetime` ranges, `template`, `faker` methods,
73 `expr`), with per-table row counts and a seed. It then executes the plan
74 deterministically: tables in dependency order, foreign keys sampled from keys
75 that exist (self-references are supported, subject to defect #8 in Section 7),
76 uniqueness enforced over single and composite key sets, CHECK constraints
77 enforced. Four profiles are available: `functional` (happy-path rows), `edge`
78 (CHECK endpoints, maximum-length strings, NULLs in nullable columns,
79 every enum value), `negative` (rows that each violate exactly one constraint,
80 tagged with the violation) and `volume`. An `auto` mode infers a plan from
81 the DDL with no LLM at all. Cost is one LLM call regardless of row count;
82 the model controls distributions only through the plan vocabulary.

83 2.3. The author’s position

84 SynthData is the author’s tool. The study therefore gives every arm
85 identical inputs: the same DDL, the same frozen business case, the same
86 row targets, the same seeds. Every arm is scored with the same database
87 and scorer. The tool version was frozen before any measured run (version
88 1.2.2, frozen 12 September 2026, before the first measured E1 cell). Defects
89 found afterwards are reported as results rather than fixed, and everything is
90 released. Section 8 returns to this.

91 3. Study design

92 3.1. Research questions

93 **RQ1 (fidelity and memorisation)** How close is LLM-provisioned data to
94 the fixtures the projects ship, and how much of the resemblance survives

¹@vijaypjavvadi/synthdata, MIT licence. The study freezes version 1.2.2; file check-sums are in the artifact.

95 when the schema’s identifiers are renamed?

96 **RQ2 (validity and cost)** Does plan-then-execute produce data that satis-
 97 fies the schema’s declared constraints more reliably than direct emis-
 98 sion, and at what cost?

99 **RQ3 (downstream)** Given the same LLM test generator, does provisioning
 100 constraint-valid fixtures raise the usable-test yield or the executable
 101 mutation kill rate, compared with letting the generator invent its own
 102 data?

103 **RQ4 (provider)** Do the answers hold across two model families?

104 RQ1 is answered by experiment E1 (fidelity) and its control E1b, RQ2 by
 105 E1 (validity and cost), RQ3 by E2, and RQ4 by running every LLM arm on
 106 Claude Sonnet 4.6 and GPT-4o.

107 3.2. Subjects for E1 and E1b

108 Table 1 lists the five schemas. Each ships human-written sample data un-
 109 der an open licence and loads into PostgreSQL 16 with constraints enabled.
 110 Their shapes differ. Pagila (film rental) has 15 base tables after collapsing
 111 the payment partitions, an enum type and many-to-many bridges. Chinook
 112 (music store) has a self-referencing employee hierarchy and a composite-key
 113 bridge. Northwind (trading) has a self-reference, a composite primary key
 114 and two empty tables. `employees` (HR history) has 3.9 million rows, a
 115 CHECK constraint, a UNIQUE text column and date-range assignments.
 116 Dell DVD Store 2 (retail) has wide customer rows and no declared for-
 117 eign keys between `orders`, `orderlines` and `products`. Pagila, Chinook and
 118 Northwind are widely mirrored with their sample data; `employees` is dis-
 119 tributed with its data as a MySQL test database; Dell DVD Store 2 ships a
 120 generator rather than a fixed dataset and is the least likely of the five to have
 121 been seen with its rows. We do not know the providers’ training sets, which
 122 is why the control in Section 3.6 exists. All five were pinned to a commit or
 123 tag and their files checksummed before any generation.

124 *Canonical DDL.* Each schema was exported from the PostgreSQL cata-
 125 logue into one canonical file with inline PK, FK, UNIQUE, CHECK and
 126 NOT NULL clauses, enum types rewritten as `CHECK (col IN (...))`, par-
 127 titions collapsed and one non-portable column (Pagila’s `tsvector`) dropped.

Table 1: E1 subjects. Human rows are the shipped fixture counts; row targets for the generated arms are $\min(\text{human}, 500)$ per table.

Schema	Pinned source	Tables	Human rows	Notable constraints
Pagila	pagila v3.1.0	15	46,273	enum, 20 FKs
Chinook	chinook-database 7f67772	11	15,607	self-ref FK, composite PK
Northwind	northwind_psql cd0ef28	14	3,362	self-ref FK, composite PK, varchar(2)
employees	test_db e324b56	6	3,919,015	CHECK, UNIQUE text, char(4) keys
Dell DVD Store 2	postgresDBSamples 2bdc953	8	172,716	no FK on order lines

128 Reloading the human fixtures into the canonical schema gives identical row
129 counts and zero rejections on all five.

130 *Business cases..* One text per schema, written from the project’s public doc-
131 umentation only (never from the fixture data) and frozen with an MD5 be-
132 fore any run. Every arm receives the same case verbatim, plus the same
133 row-target line.

134 *Row targets..* $\min(\text{human count}, 500)$ per table, chosen after timing the di-
135 rect arm at roughly 100 seconds per 100 rows. The cap is conservative against
136 the plan arm: the direct arm’s dominant failure, repeated keys across chunks,
137 grows with the number of rows, so a higher cap would widen the validity gap
138 in the engine’s favour. Fidelity metrics compare a 500-row sample with the
139 full human distribution.

140 3.3. Arms for E1

- 141 1. **Direct emission.** The model receives the canonical DDL, the business
142 case, the table to fill, its exact row target, a sample of up to 60 already-
143 emitted parent keys per foreign-key column, and the last 30 keys it
144 emitted for the same table; it must answer with CSV only. Chunks of
145 200 rows; temperature 0; every call’s tokens and latency are logged.
- 146 2. **Plan-then-execute.** SynthData’s own system prompt and user-message
147 format, temperature 0, one call; the plan is executed with seed 42 under
148 the **functional** profile. The **edge** and **negative** profiles are executed
149 from the run-1 plans without further LLM calls.
- 150 3. **Auto.** SynthData’s no-LLM mode, seed 42.
- 151 4. **Faker.** A plain Faker [3] script per schema with no referential resolu-
152 tion: what a developer gets from the ecosystem default.
- 153 5. **Human.** The shipped fixtures, unmodified.

154 LLM arms run on both providers with two independent runs each: 40 LLM
155 cells in total.

156 3.4. *Scoring validity*

157 For each cell, a fresh database is created from the canonical DDL and the
158 emitted CSVs are inserted row by row in foreign-key order inside a PL/pgSQL
159 loop that catches every exception and records the table, row and SQLSTATE.
160 Rows that violate anything are rejected, so a rejected parent makes its chil-
161 dren fail their foreign key, which is the real cost of an invalid row. A retry
162 pass re-attempts rows rejected only for foreign-key reasons until no progress is
163 made, so that within-table order (a self-referencing hierarchy emitted child-
164 before-parent) is never counted against an arm. Casts use base types so
165 that declared lengths and precisions are enforced by the insert. SQLSTATE
166 classes 23503, 23505, 23514, 23502 and 22xxx are reported as FK, UNIQUE,
167 CHECK, NOT NULL and type/length. The scorer was validated on the hu-
168 man fixtures (100% load on all five schemas) and on a Chinook copy with
169 one planted violation of each class, which reports exactly the five planted
170 rows plus their foreign-key cascade.

171 3.5. *Scoring fidelity*

172 Per column, against the human fixture of the same schema. Categorical
173 columns are of two kinds. *Declared-domain* columns are enum, CHECK-IN
174 and boolean columns, whose legal values are in the DDL. *Free-text* columns
175 are text columns with at most 30 distinct human values, such as city or job
176 title, whose values are not in the DDL. For both we compute the Jensen-
177 Shannon distance (base 2; 0 identical, 1 disjoint) and the fraction of human
178 values present. Numeric and date columns get the two-sample Kolmogorov-
179 Smirnov statistic and the fraction of the human range covered; foreign-key
180 columns the absolute difference in fan-out Gini; nullable columns the null-
181 rate difference. Aggregates are medians over a schema’s columns; surrogate
182 keys are excluded. The declared-domain versus free-text split matters for
183 RQ1: an arm can match a declared domain by reading the DDL, but it can
184 match the distribution of a free-text column only by knowing the data.

185 3.6. *E1b: the renamed-schema control*

186 A deterministic renamer replaces every table and column identifier (54 ta-
187 ble names and 202 distinct column names) with a synonym (`customer`→`client`,

188 `order→purchase`, `film→movie`, `us_states→state_codes`, and so on), re-
 189 moves the dataset names and documentation references from the business
 190 cases, and applies the same synonyms to the case prose. Types, constraints,
 191 CHECK value lists, row targets, prompts and seeds are unchanged; the hu-
 192 man fixtures reload into the renamed DDLs at 100 %, and the engine’s `auto`
 193 mode reaches 100 % on all five. The LLM arms are rerun once per provider
 194 (20 cells) and each renamed cell is paired with the run-1 cell of the same
 195 schema, provider and arm. Two references calibrate the effect: the run-to-
 196 run variation of the original schemas (run 1 versus run 2) bounds what a
 197 rerun alone can change, and the least-mirrored schema (Dell DVD Store 2)
 198 bounds what renaming costs a model that has nothing to recall.

199 3.7. E2: provisioned data downstream

200 E2 reuses the RAITG pipeline of [2], whose requirements, mutants, har-
 201 ness and baseline run logs are archived on Zenodo as RAITG v5.0.0, with
 202 its scoring protocol unchanged. The pipeline covers 172 requirements over
 203 four community Python services (`httpbin`, `fastapi-restful`, `flaskr`, `fastapi-task-`
 204 `manager`). Tests are generated per requirement with the grounded prompt
 205 (SUT source injected, verification and repair on), extracted into per-service
 206 pytest harnesses, and filtered to *known-good* units, those that pass on clean
 207 source. The known-good units are scored against 293 frozen mutants; a kill
 208 is a new failure or collection error. Three services have a database; `httpbin`
 209 has none and serves as a negative control.

210 For the three database-backed services we derived a DDL (`flaskr`’s `schema.sql`;
 211 the SQLAlchemy `Task` model; the `players` table of the shipped SQLite file),
 212 wrote a business case from each project’s README and REST examples,
 213 and generated fixtures with SynthData 1.2.2 (Claude plan, seed 42, three
 214 profiles). Each fixture was validated through the application itself: logins
 215 succeed and list endpoints return the fixture. The RAITG prompt (v1.5.0)
 216 gains one optional element, a [TEST DATA] block; without it the prompt is
 217 byte-identical to the published v1.4.0. Four arms:

218 **G** the grounded arm of [2] (baseline). For Claude these are the archived run
 219 logs re-scored under the E2 harness; for GPT-4o they are new runs,
 220 since no archived GPT-4o logs exist.

221 **G+D-func** the functional fixture shown to the model and pre-loaded into
 222 the service’s database at scoring.

223 **G+D-all** the same pre-loaded fixture, plus the **edge** rows shown as valid
224 boundary payloads and the **negative** rows shown as invalid payloads
225 with their violation tags. The database state is identical to G+D-func,
226 so the difference isolates the information given to the model.

227 **G+D-human** the project’s own fixture shown (flaskr’s two users and one
228 post; the 26 shipped players; the task manager ships none).

229 **G+D-state** added after the first four arms were scored (E2c): the G+D-
230 func block followed by an explicit database-state contract, stating that
231 the tables are never empty, that ids $1..N$ are taken, that the first row a
232 test creates receives id $N + 1$, that count assertions must be relative to
233 N , that the listed rows are the ones to use for existing-row tests, and
234 (for fastapi-restful) that squad numbers 1–26 are taken. The database
235 state at scoring is identical to G+D-func; only the information given
236 to the model differs. Run once, for Claude, against the run-1 baseline.

237 At scoring time, each arm’s conftest provisions its fixture wherever the database
238 is initialised: the default instance database, the harness fixtures, and any
239 database a generated test creates itself through the application’s `init_db()`
240 or `Base.metadata.create_all()`. The promise made in the prompt there-
241 fore holds in the run. Two providers, two runs each for the first four arms.
242 The hypothesis, written in the research plan on 11 September 2026 before
243 any E2 run (the plan is in the artifact), was that the data block raises usable
244 yield on the database-backed services and leaves httpbin unchanged, with
245 any kill-rate gain concentrated on validation-path mutants. The E2c arm
246 was designed after that hypothesis had been rejected, and its own expecta-
247 tions were written down before it ran (also in the artifact): that the state
248 contract returns usable yield to the baseline level (aggregate $\geq 88\%$, task
249 manager $\geq 90\%$), that the kill count stays within ten of the baseline, and
250 that httpbin moves by at most seven.

251 3.8. Statistics

252 E1 and E1b comparisons are paired over cells (E1: schema $\times$ provider $\times$
253 run; E1b: renamed versus original per cell) and reported as the paired median
254 difference with a bootstrap 95 % confidence interval (10,000 resamples, seed
255 42), Cliff’s delta, and the Wilcoxon signed-rank p -value. Cells that share a
256 schema are not independent, so we also report the schema-level result ($n = 5$)

as a sensitivity check. E2 comparisons are paired over mutants (the 293 per-mutant kill vectors of an arm versus its baseline in the same provider and run) with the exact McNemar test, a bootstrap 95 % confidence interval of the difference (1,000 resamples of mutants, seed 42), and a Holm correction over the twelve tests. Runs are reported side by side, never pooled.

4. RQ1: fidelity, and what survives renaming

4.1. Fidelity before the control

On the surface, direct emission is far closer to the human data than plan-then-execute. Over the 18 paired cells in which both arms produced data, the median categorical Jensen–Shannon distance is 0.083 for direct emission and 1.000 for the plan arm ($p = 0.003$, Cliff’s $\delta = 0.51$); the fraction of human categorical values reproduced is 1.00 versus 0.00 ($p = 0.004$). Numeric and date distributions do not differ (KS 0.522 versus 0.501, $p = 0.83$); plans cover the human numeric ranges better (0.960 versus 0.520, $p = 0.019$); emitted rows imitate the human foreign-key fan-out slightly better (Gini difference 0.068 versus 0.101, $p = 0.028$).

Splitting the categorical columns changes the picture. On declared-domain columns, which exist only in Pagila among the five schemas (the rating enum and two booleans; four cells), both arms match the human distributions: median JS 0.043 for the plan arm and 0.110 for direct emission. The entire categorical gap is in free-text columns (median 1.000 versus 0.083, identical to the aggregate). Two further observations point the same way. With Claude, the direct arm scores a JS distance of exactly 0.000 on 22 of the 36 categorical columns of Northwind and on 8 of the 11 of Pagila, in both runs (median 0.000): on those columns it re-emits the published dataset value for value. And on Dell DVD Store 2, the schema least likely to have been seen with its data, the direct arm’s JS distance (0.41 with Claude, 0.88 with GPT-4o) and value coverage (0.88, 0.22) are no better than the plan arm’s (0.47, 0.86 and 0.60, 0.19).

4.2. The renamed-schema control

Table 2 and Fig. 1 pair each renamed direct-emission cell with its original run-1 cell.

Table 2: E1b: direct-emission cells, original versus renamed schema (run 1). Load rate in %; JS = median categorical Jensen–Shannon distance to the human data; cov = fraction of human categorical values reproduced; Δ JS = renamed – original.

Schema	Provider	load orig	load ren	JS orig	JS ren	Δ JS	cov orig	cov ren
Chinook	Claude	93.8	99.2	0.000	0.326	+0.33	1.00	0.80
Chinook	GPT-4o	99.7	99.7	1.000	1.000	0.00	0.00	0.00
Northwind	Claude	98.4	97.2	0.000	0.263	+0.26	1.00	0.95
Northwind	GPT-4o	77.3	87.8	0.587	1.000	+0.41	0.63	0.00
Pagila	Claude	99.9	99.6	0.000	1.000	+1.00	1.00	0.00
Pagila	GPT-4o	94.7	96.1	1.000	1.000	0.00	0.00	0.00
employees	Claude	99.8	95.4	0.054	0.085	+0.03	1.00	1.00
employees	GPT-4o	98.6	98.7	0.083	0.389	+0.31	1.00	1.00
Dell DVD Store 2	Claude	100	97.7	0.406	0.589	+0.18	0.88	0.83
Dell DVD Store 2	GPT-4o	100	100	0.875	0.875	0.00	0.22	0.22
median		99.2	98.2	0.245	0.732	+0.22	0.94	0.51

289 *Validity is unaffected..* Direct-emission load rates do not change under re-
290 naming (median 99.2 % versus 98.2 %; paired median difference 0.000, 95 %
291 CI $[-0.012, +0.028]$; $p = 1.0$). The models’ ability to satisfy constraints does
292 not depend on recognising the dataset, which also rules out the alternative
293 reading that the renamed schemas are simply harder to understand.

294 *The fidelity advantage does not survive..* The direct arm’s median categorical
295 JS distance rises from 0.245 to 0.732 (paired median difference +0.22, 95 % CI
296 $[0.00, +0.36]$; $p = 0.016$), its value coverage falls from 0.94 to 0.51 ($p = 0.062$),
297 and its numeric/date KS statistic rises from 0.522 to 0.602 (median difference
298 +0.06, CI $[0.00, +0.09]$; $p = 0.023$). Two references put these deltas in scale.
299 The run-to-run variation of the same metric on the original schemas (run 1
300 versus run 2, ten schema–provider pairs) has median 0.000 and maximum
301 0.056, so a rerun alone does not move the distance. And on Dell DVD
302 Store 2, the least-mirrored schema, renaming costs Claude 0.18 and GPT-
303 4o 0.00, which bounds what synonyms cost a model with nothing to recall.
304 On the three widely mirrored schemas Claude’s deltas are +0.33 (Chinook),
305 +0.26 (Northwind) and +1.00 (Pagila), and GPT-4o, which was already at
306 JS 1.0 on Chinook and Pagila, moves +0.41 on Northwind, the one it had
307 partly reproduced. Claude’s exact reproduction of Pagila (JS 0.000, coverage
308 1.0) becomes a disjoint distribution (JS 1.000, coverage 0.0).

309 We read this as memorisation, with the stated caveats: the effect is
310 far above run-to-run noise and above the renaming cost on the unfamiliar
311 schema, and it appears where the published data is famous and the model

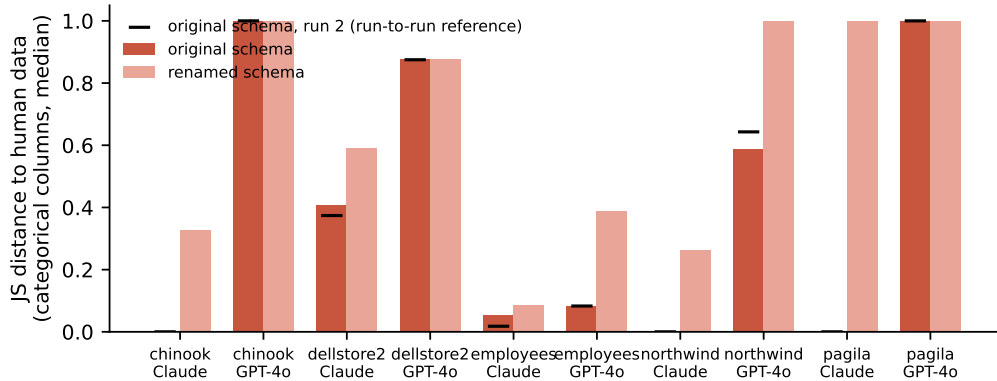


Figure 1: E1b: categorical distance of directly emitted data to the human fixtures, original versus renamed identifiers (direct arm, run 1). The black tick on each original bar is the same cell’s run-2 value, i.e. the run-to-run reference (median $|\Delta|$ 0.000, maximum 0.056). Claude’s exact reproduction of Northwind, Chinook and Pagila disappears when the names change; Dell DVD Store 2, the least-mirrored schema, moves by 0.18 (Claude) and 0.00 (GPT-4o).

312 had reproduced it exactly. Under renamed schemas the direct arm’s categor-
 313 ical fidelity (median 0.73) is much closer to the plan arm’s (median 1.0) than
 314 the 0.08-versus-1.00 comparison of Section 4.1 suggested. The RQ1 answer is
 315 therefore: on an unseen schema neither approach reproduces the categorical
 316 distributions of the human data; the plan arm covers numeric ranges better;
 317 the direct arm imitates fan-out slightly better; and the large categorical ad-
 318 vantage of direct emission on public schemas is recall of Northwind, Sakila
 319 and Chinook.

320 One cue survives renaming. Claude’s renamed-Pagila plan still placed
 321 rental dates in 2005–2006, the Sakila era, so structural recognition is not
 322 fully removed by identifier renaming. The control removes lexical identity,
 323 not structure; a stronger control would perturb values as well, or use a schema
 324 published after the providers’ training cut-offs.

325 *The plan arm under renaming..* Plan-arm load falls from a mean of 0.888
 326 (run 1) to 0.711, and every sub-100 renamed cell is again defect #6 or #7
 327 (Section 7): with synonym names both models reached for `expr` date arith-
 328 metic in 4 of 10 plans (3 of 20 in E1), and the engine handles it wrongly. The
 329 plan arm’s validity ceiling is set by the engine’s handling of LLM-written
 330 plans, not by the model.

Table 3: E1 load rate (% of emitted rows accepted by PostgreSQL with all constraints on), run 1 / run 2. “abort” = the engine aborted on the LLM-written plan (defect #6) and produced no data, counted as 0.

Schema	plan-then-execute		direct emission		auto	Faker	human
	Claude	GPT-4o	Claude	GPT-4o	(no LLM)		
Chinook	100 / 100	100 / 100	93.8 / 98.1	99.7 / 99.6	100	99.3	100
Northwind	100 / 100	abort / abort	98.4 / 97.7	77.3 / 77.1	100	23.5	100
Dell DVD Store 2	93.8 / 100	93.9 / 93.8	99.97 / 98.8	100 / 99.97	100	100	100
Pagila	100 / 100	100 / 90.8	99.9 / 99.6	94.7 / 94.4	100	52.3	100
employees	100 / 100	100 / 100	99.8 / 99.3	98.6 / 97.7	100	0.4	100

5. RQ2: validity and cost

5.1. Validity

Table 3 and Fig. 2 give the load rate of every cell. Over the 20 paired (schema, provider, run) cells the plan arm’s mean is 0.886 with median 1.000 and the direct arm’s mean is 0.962 with median 0.987. The two engine aborts (defect #6, Northwind with GPT-4o, both runs), counted as zero, put the direct arm ahead on the mean by 0.076; on the paired median difference the arms are level (+0.004, 95 % CI $[-0.017, +0.015]$; Wilcoxon $p = 0.85$; schema-level $p = 0.81$). Excluding the two aborts ($n = 18$) the means are 0.985 and 0.983 and the plan arm is marginally ahead (paired median difference +0.006, CI $[+0.002, +0.017]$; Cliff’s $\delta = 0.53$; $p = 0.29$). We state the result as: at 500 rows per table on these schemas, plan-then-execute has no validity advantage over direct emission, and a user who counts the aborts sees it slightly behind. The difference between the arms is in how they fail.

The plan arm is bimodal: 14 of the 18 executed cells load at exactly 100 %, and each of the remaining four is attributable to one engine defect visible in the plan and the rejection log. Three Dell DVD Store 2 cells lose 185–187 rows to duplicate primary keys because the model wrote `{gen: fk, ref: inventory.prod_id}` for a primary-key column and the engine sampled the reference with replacement (defect #7; the schema declares no foreign key there, which is why the model reached for one). One Pagila cell loses 443 rows because an `expr` generator received its date operand as a string and concatenated instead of adding (2025-03-18 16:25:007; defect #6), and the two Northwind GPT-4o cells abort on the same defect. Across all executed plan cells the engine emitted zero foreign-key, CHECK and NOT NULL violations; its 559 UNIQUE and 443 type rejections are the two defects.

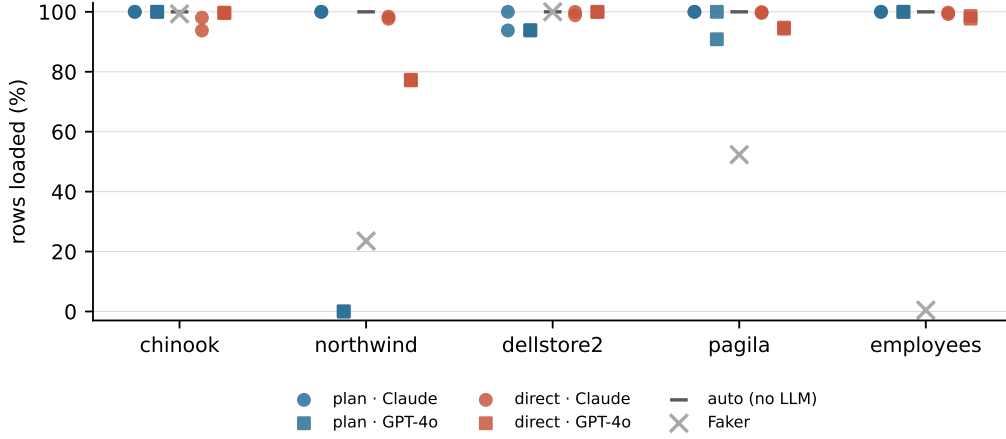


Figure 2: E1 load rate per schema and arm (both providers, both runs). The plan arm is at 100 % except where an engine defect bites; direct emission degrades with schema difficulty and model.

357 Direct emission is never 100 % on a schema with declared foreign keys
 358 (its one 100 % is Dell DVD Store 2, which declares none), and its rejections
 359 are the model’s: 504 foreign-key, 1,009 uniqueness, 4 NOT NULL and 79
 360 type/length violations over 20 cells. The effect grows with schema difficulty
 361 and shrinks with model strength: on Northwind, GPT-4o loads 77.3 and
 362 77.1 % in the two runs (152/139 foreign-key, 147/164 uniqueness and 13/12
 363 type rejections), Claude 98.4 and 97.7 %. Repeated keys across chunks are
 364 the dominant failure: the model is told the last 30 keys it emitted and still
 365 repeats keys from earlier chunks.

366 The baselines frame the result. The engine’s no-LLM **auto** mode loads
 367 100 % on all five schemas. Validity, on these schemas, comes from the en-
 368 gine; the LLM’s contribution to the plan arm is the plan’s semantics, which
 369 Section 4 measures as fidelity. The Faker script, the ecosystem default, loads
 370 100 % only where every foreign key happens to point at a table whose keys
 371 are 1 . . . N (Dell DVD Store 2, Chinook) and collapses elsewhere (Northwind
 372 23.5 %, Pagila 52.3 %, **employees** 0.4 %): the referential engine is what a
 373 Faker script lacks.

374 *Profiles..* Both were executed from the run-1 plans without further LLM
 375 calls. The **edge** profile loads at 100 % on every schema except Dell DVD
 376 Store 2 (93.8 %, defect #7 again). The **negative** profile loads at 0 % on

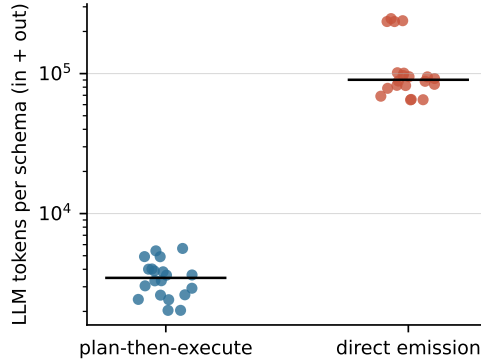


Figure 3: LLM tokens per schema cell (in + out, log scale; bars are medians): 3,472 for plan-then-execute versus 90,340 for direct emission.

every schema, which is the property a negative fixture must have: each of its rows is rejected for a stated class (type, NOT NULL, foreign key or CHECK) that the scorer reports.

5.2. Cost

Per cell (Fig. 3), the plan arm uses a median of 3,472 tokens in one call and the direct arm 90,340 tokens in 14–45 calls; the median of the per-cell ratios is 29.5 (the ratio of the medians is 26.0). Wall-time medians are 15 s and 658 s (per-cell median ratio 46; the longest direct cell, Pagila with Claude, took 28 minutes). Over the study the plan arm consumed 40k input and 31k output tokens, the direct arm 865k and 1,441k.

6. RQ3: provisioned data and LLM-generated tests

6.1. Harness

Two harness issues surfaced during scoring and were fixed before any number was final; both are recorded because the second changes a baseline. First, the fixture’s password hashing (werkzeug’s default 260k PBKDF2 rounds, re-run before every test and on every `init_db()` call) pushed the flaskr suite past the scorer’s 90-second limit, which the RAITG protocol counts as a kill. Hashes are therefore memoised with a low iteration count; logins are unaffected, since the verifier reads the method from the stored hash. Second, one GPT-4o flaskr unit calls the application’s `init_db_command()` at module level. The resulting `SystemExit` inside pytest’s collection aborted every

Table 4: E2: mutants killed (of 293) and usable yield (known-good units / emitted units) per arm, provider and run.

Arm	Claude r1	Claude r2	GPT-4o r1	GPT-4o r2
G (grounded RAITG)	161 (88 %)	169 (89 %)	106 (52 %)	109 (59 %)
G + SynthData functional fixture	158 (85 %)	167 (84 %)	115 (53 %)	109 (49 %)
G + functional + edge/neg. payloads	170 (83 %)	167 (85 %)	106 (50 %)	113 (48 %)
G + project’s own fixture	145 (87 %)	167 (81 %)	109 (51 %)	102 (48 %)
G + functional fixture + state contract (E2c)	160 (93 %)	—	—	—

Table 5: E2 paired versus G, same provider and run: difference in mutants killed, McNemar exact p , Holm-adjusted p over the twelve pre-planned tests, bootstrap 95 % CI. The E2c arm was tested afterwards and is reported unadjusted.

Provider	Arm	r1 Δ	p	p_{Holm}	CI	r2 Δ (p)	CI
Claude	functional	−3	0.51	1.0	[−9, +3]	−2 (0.75)	[−8, +4]
Claude	func + edge/neg	+9	0.022	0.25	[+2, +16]	−2 (0.73)	[−7, +3]
Claude	project fixture	−16	<0.001	0.005	[−25, −8]	−2 (0.82)	[−11, +6]
Claude	func + state contract (E2c)	−1	1.0	—	[−6, +3]	—	—
GPT-4o	functional	+9	0.19	1.0	[−2, +22]	0 (1.0)	[−9, +11]
GPT-4o	func + edge/neg	0	1.0	1.0	[−12, +11]	+4 (0.57)	[−6, +14]
GPT-4o	project fixture	+3	0.63	1.0	[−5, +11]	−7 (0.19)	[−16, +1]

398 pytest process for that service. The original group-level known-good filter
 399 could not attribute the failure to a unit and kept all 130, and the scorer then
 400 saw the same exit status on the baseline and on every mutant, that is, zero
 401 kills. The E2 harness therefore adds a per-unit pre-screen before the known-
 402 good filter: each unit runs alone under a 30-second wall cap (the limit the
 403 RAITG protocol already applies per test function, which pytest’s timeout
 404 cannot apply to module-level code), and units that cannot pass alone are
 405 excluded and recorded. The pre-screen removes units; it cannot add kills.
 406 Every arm, including the baseline for both providers, was re-scored under
 407 this one harness before any comparison was made. The Claude baseline logs
 408 re-scored under it give 161 and 169 kills against the archived 160 and 159;
 409 the difference is three fastapi-restful units that pass only after an earlier unit
 410 has warmed the service’s ten-minute response cache, order-dependent units
 411 that the stricter filter removes. No E2 chunk contains a timeout-based kill.

412 6.2. Usable yield and kill rate

413 Table 4 and Figs. 4–5 give the aggregate results; Table 5 the paired statis-
 414 tics.

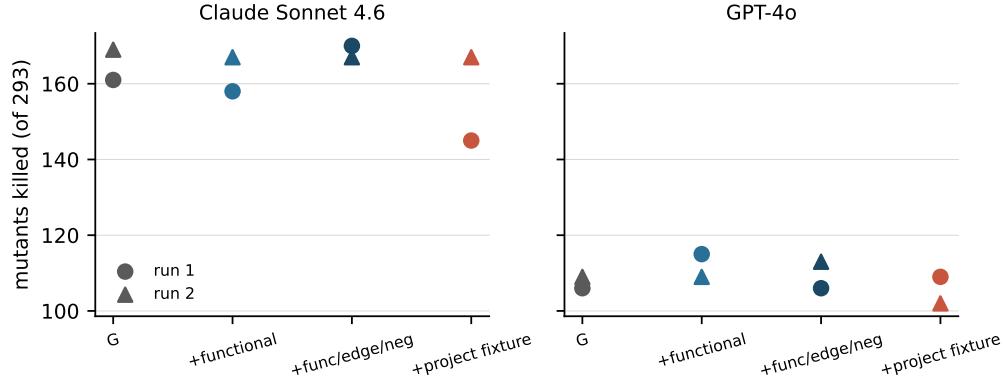


Figure 4: E2: mutants killed per arm (circles run 1, triangles run 2). No arm moves the kill rate reliably.

415 *The advance hypothesis is rejected..* A provisioned fixture does not raise us-
 416 able yield. For Claude it lowers it, from 88–89 % to 83–85 %, and most on
 417 the task manager (known-good units 241 of 254 in G, 176 of 275 with the
 418 functional fixture). The failing units explain why: with 20 tasks pre-loaded,
 419 generated tests that create one task and assert a count of one, or that assume
 420 the row they created has id 1, fail on clean source, even though the prompt
 421 lists the pre-loaded rows and their ids. Provisioned rows conflict with the
 422 empty-database assumption the generator carries, and listing the rows does
 423 not remove the conflict.

424 *Stating the contract does (E2c)..* The G+D-state arm adds to the same func-
 425 tional block an explicit statement of what the pre-loaded rows imply for a
 426 test: ids 1.. N are taken, a created row receives $N + 1$, counts are relative to
 427 N . With the database state unchanged, usable yield returns to and slightly
 428 above the baseline: 1,207 of 1,294 units (93.3 %) against 87.9 % for G and
 429 84.9 % for G+D-func, and on the task manager 271 of 281 (96.4 %) against
 430 241 of 254 (94.9 %) and 176 of 275 (64.0 %). The generated tests read the
 431 contract: the failing patterns of G+D-func (assert a count of one; assume
 432 id 1) are replaced by tests that capture the id from the create response and
 433 assert relative counts or the presence of the row they created. The kill count
 434 is 160 against the baseline’s 161 (paired difference -1 ; 2 arm-only and 3
 435 baseline-only mutants; McNemar $p = 1.0$; bootstrap CI $[-6, +3]$), with http-
 436 bin at $+1$, fastapi-restful and flaskr unchanged, and the task manager at

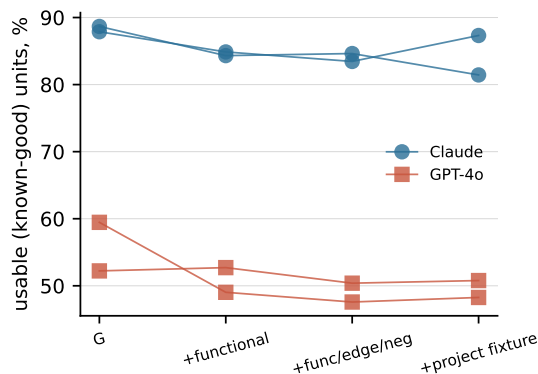


Figure 5: E2: usable yield per arm. Provisioned fixtures lower the share of generated tests that pass on clean source for Claude.

36 against 38 (32 with the functional block alone). All three expectations
written down before the run hold. The block costs 16 % more input tokens
than G (1.44M against 1.24M), about the same as the functional block alone.
The yield loss of provisioning is therefore a fixture-contract problem that the
prompt can solve; solving it does not change what the tests detect.

No arm reliably changes the kill rate.. Of twelve paired comparisons, two
are nominally significant and neither replicates. Claude with the bound-
ary/negative payloads gains 9 mutants in run 1 ($p = 0.022$; Holm-adjusted
 $p = 0.25$), driven by fastapi-restful (16→25), the validation-path mutants
(409 on a duplicate squad number, 422 on an invalid payload) that the ad-
vance hypothesis named; in run 2 the baseline itself kills 25 on that service
and the arm adds nothing. Claude with the project’s fixture loses 16 in run
1 ($p < 0.001$; Holm-adjusted $p = 0.005$), a flaskr collapse (31→15, with 113
of 163 units usable) that does not recur in run 2 (21). A 9-mutant change
is 3 percentage points of the 293. GPT-4o’s differences are within ± 9 with
17–37 discordant mutants per comparison. The minimum difference the ex-
act test can detect at these discordant counts is roughly 6–12 mutants, and
the run-to-run variation of the baseline itself is of that size (161 versus 169
for Claude). httpbin, the service with no database and no block, moves by
at most seven mutants (between +1 and +7 for Claude, between −5 and +2
for GPT-4o), inside the run-to-run band, as a control should. Two runs are
not enough to separate a ten-mutant arm effect from run-to-run variation on

459 this harness, and we report both runs rather than the favourable one.

460 *Provider (RQ4)*.. GPT-4o is a different regime: about three tests per re-
461 quirement (516 units per run against Claude’s 1,300), half of them unusable,
462 three to five times more repair passes, and 102–115 kills whatever the arm.
463 The RAITG prompt was developed against Claude, so the two providers are
464 not compared as equals; RQ4 is answered as “the null replicates”, not “the
465 providers are equivalent”.

466 *Cost*.. The functional block adds 14 % input tokens per condition run (Claude
467 1.24M→1.41M), the functional block with the state contract 16 % (1.44M),
468 the full block 40 % (1.74M); output tokens change by less than 10 % and
469 repair counts do not rise.

470 7. The defect trail

471 An executable study against a tool finds defects in the tool. We fixed the
472 ones found before the study version was frozen and report the ones found
473 afterwards as results. Five were found and fixed before freezing at 1.2.2.
474 Numeric(p, s) precision was ignored (Pagila `rental_rate` received 752.09;
475 199 of 200 film rows were rejected, with a cascade into four child tables). A
476 composite PRIMARY KEY was not treated as a uniqueness set (305 duplicate
477 bridge rows on Chinook). A UNIQUE text column with a small fake-value
478 pool aborted generation. `varchar(n)` was not enforced for plan-driven gen-
479 erators (a three-character state abbreviation went into `varchar(2)`). And
480 `{gen: fk}` on a column without a declared foreign key aborted the run. The
481 v1.1.0 smoke numbers that motivated these fixes (Pagila 60.1 %, `employees`
482 aborting) are in the artifact’s cell table. Four were found after freezing, by
483 LLM-written plans. #6: `expr` receives dates as strings (seven cells across E1
484 and E1b). #7: `fk` on a primary-key column samples with replacement (E1
485 and E1b, Dell DVD Store 2). #8: a self-referencing `fk` on a NOT NULL col-
486 umn emits NULL for the first rows (found while validating the E2 fixtures; no
487 E1 cell is affected). #9: the YAML parser rejects a multi-line flow sequence
488 that PyYAML accepts (the E2 task-manager plan, twice at temperature 0;
489 re-serialised mechanically for the fixture and recorded). Every plan-arm miss
490 in E1 and E1b is #6 or #7. The plan arm’s validity ceiling is the engine’s
491 plan handling, and a user of version 1.2.2 inherits these defects.

492 8. Threats to validity

493 *Construct..* Validity is measured as acceptance by PostgreSQL with declared
494 constraints; undeclared business rules (an order line for a discontinued prod-
495 uct) are invisible to it, and the human schemas declare almost no CHECK
496 constraints, so the constraint surface is dominated by keys and nullability. Fi-
497 delity metrics are column-wise and blind to cross-column coherence (a player
498 whose position and abbreviation disagree, as SynthData’s E2 fixtures do);
499 the fixture arms in E2 test provisioning, not realism, and the project-fixture
500 arm is the realism control. Free-text fidelity can only be achieved by know-
501 ing the data, which is why we report declared-domain and free-text columns
502 separately. In E2, a kill is a failing or crashing unit; script-style units bypass
503 pytest’s per-test timeout, which is why the pre-screen exists.

504 *Internal..* The direct arm is the most favourable naive design we could build
505 (parent keys shown, chunked, resumable); a weaker one would only widen
506 the gap. The 500-row cap favours direct emission (Section 3). Temperature
507 0 is not determinism: the same prompt produced plans that parsed and
508 plans that did not, and run-to-run differences of ten mutants. The harness
509 hardening in E2 was applied uniformly after discovery, and both scorings
510 of the Claude baseline are on record. The E2 fixture’s flaskr passwords are
511 stored with a low PBKDF2 iteration count for timing reasons; login semantics
512 are unchanged.

513 *Conclusion..* E1 has 20 cells over five schemas and E1b ten pairs; cells shar-
514 ing a schema are not independent, and the schema-level test ($n = 5$) is
515 reported alongside. E2’s two runs give a minimum detectable effect of 6–12
516 mutants, the size of the baseline’s own run-to-run variation. Twelve E2 tests
517 are Holm-corrected; the E2c arm is a single run added afterwards, tested
518 against expectations written before it ran, and reported unadjusted.

519 *External..* Five schemas, four services, two providers, 500 rows per table.
520 Memorisation is only partly controlled: renaming removes lexical cues, not
521 structural ones (Section 4.2). GPT-4o’s low unit yield confounds the provider
522 comparison in E2. The E2 baseline is asymmetric (published Claude logs,
523 fresh GPT-4o runs).

524 *Conflict of interest..* The tool is the author’s. The mitigations are identical
525 inputs and scoring for every arm, a version frozen before the first measured

run, an independent no-LLM and a Faker baseline, published defects, and a released artifact that regenerates every number. The paper’s central claims are that validity comes from the engine rather than the model, that the fidelity advantage belongs to the competing approach and is recall, and that the downstream hypothesis is rejected; none of these favours the tool.

9. Related work

LLMs as test-data generators.. Baudry et al. [4] asked LLMs either to emit test data directly or to write test-data generator code, across languages and cultural contexts, and evaluated domain adequacy and executability; their generator-writing mode is the nearest precedent for plan-then-execute, and their evaluation did not check relational constraints or compare against human fixtures. Kannan et al. [5] generated test data for complex SQL schemas with an LLM in a privacy-restricted industrial setting and judged structural and semantic validity with schema checks and an LLM-as-a-judge; our scorer replaces the judge with the database. Neither study controlled for the possibility that the model had seen the data. Deep generative models for tabular data, from conditional GANs [6] to fine-tuned language models that serialise rows as text [7], target distributional fidelity for machine-learning use. Our consumer is a test, so hard constraints are the primary requirement. And our data is requested for a public schema rather than sampled from a private table, which is what raises the recall question.

Test data for database applications.. AGENDA [8] populated databases from schema constraints and user-supplied value sets; later work generated databases to satisfy declarative constraints and cardinality targets [9], produced realistic values at scale for benchmarking [10, 11], derived inputs from the queries a program issues [12], and defined coverage criteria over SQL predicates [13]. Search-based generation of test data for a schema’s integrity constraints [14, 15] is the closest ancestor of the engine studied here; SynthData’s **negative** profile is a generator-vocabulary rendering of that idea, and our scorer is the oracle those works rely on. None of this literature considered a language model as the author of the generator.

Memorisation and contamination.. That language models reproduce training data verbatim is established [16], and benchmark contamination is a recognised problem in natural-language evaluation [17, 18]. For code, ReCode [19]

560 evaluates generation models under semantics-preserving perturbations, in-
 561 cluding identifier renaming; the renamed-schema control is the same idea
 562 applied to relational fixtures, whose value is precisely that they are public
 563 and long-lived, so held-out data is not available. Its result—a fidelity advan-
 564 tage that disappears under renaming while validity does not move—is the
 565 relational counterpart of the contamination effects reported for text bench-
 566 marks, and we are not aware of a prior measurement of it for LLM-generated
 567 test data.

568 *LLM-generated tests and their execution.* LLM unit-test generation has been
 569 evaluated on coverage and correctness [20, 21, 22], compared with search-
 570 based generation [23] and combined with it [24]; surveys [25] catalogue the
 571 design space. Where these studies evaluate against mutated programs they
 572 inherit mutation analysis and its caveats about mutant validity [26, 27]; E2
 573 reuses an existing, archived executable benchmark so that the only variable
 574 is the data block. Two findings anticipate ours: generated tests often fail
 575 for environmental reasons, such as wrong fixtures and assumptions about
 576 state [20, 22], and the test-pattern literature distinguishes fresh fixtures built
 577 by each test from shared or prebuilt fixtures that tests must respect [28].
 578 LLM-generated tests, in our data, behave as if every fixture were fresh.

579 *Faker.* The de facto tool for test data in Python is Faker [3], which produces
 580 plausible scalar values with no notion of a schema; our Faker arm is that
 581 default applied per table, and its foreign-key failures on four of five schemas
 582 quantify what a relational engine adds before any LLM is involved.

583 10. Conclusion

584 The realism that LLM-emitted test data shows on well-known schemas is
 585 largely recall of the published fixtures: it disappears when the identifiers are
 586 renamed, while the model’s ability to satisfy the schema’s constraints does
 587 not change. Evaluations of LLM data generation on public schemas therefore
 588 need a renamed-schema control, and we provide one. Planning instead of
 589 emitting does not buy validity at 500 rows per table; the engine’s guarantees
 590 hold by construction and its misses are engine defects, the model’s misses
 591 are foreign-key and uniqueness violations that grow with schema difficulty,
 592 and planning costs a thirtieth of the tokens. Constraint-valid fixtures do not
 593 make an LLM test generator kill more mutants, and they reduce the share
 594 of usable tests because the generator writes for an empty database. Telling

the generator which identifiers are taken and that counts start at N restores the yield without changing the kill rate. A provisioning tool should therefore ship a fixture together with its state contract, and a test generator should expect one.

Data availability

The complete artifact (pinned subjects with checksums, canonical and renamed DDLs with rename maps, frozen business cases, row targets, the E2 DDLs, cases, fixtures and prompt blocks, every arm’s output with per-call tokens and latency, every score and metric file, the per-mutant kill vectors, known-good lists and pre-screen records of E2, the scoring harness, the cell-table builder and the scripts that regenerate every table, figure and statistic in this paper) is archived on Zenodo as **paperH-artifact v1.1.0**, DOI 10.5281/zenodo.22939052, and mirrored at github.com/javvadivijayprasad/llm-testdata-memorisation (tag **v1.1.0**, the same files). SynthData 1.2.2 is published on npm; its file checksums are in the artifact.

CRedit authorship contribution statement

Vijay Prasad Javvadi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of competing interest

The author is the developer of SynthData. See Section 8.

Declaration of generative AI use

Generative AI (Claude) was used as a coding and drafting assistant under the author’s direction; the author takes full responsibility for the content. All LLM calls that are part of the experiments are described in the paper and logged in the artifact.

References

- [1] V. P. Javvadi, SynthData: schema + business case + LLM plan $\rightarrow$ constraint-valid test data, npm package @vijaypjavvadi/synthdata, version 1.2.2, MIT licence (2026).
- [2] V. P. Javvadi, Requirements or source? What grounds LLM test generation on community-maintained open-source services: An executable mutation study against an independent human baseline, manuscript under review, 2026. Artifact (RAITG v5.0.0: requirements, frozen mutants, run logs, scoring harness) archived on Zenodo, concept DOI 10.5281/zenodo.20285103, version DOI 10.5281/zenodo.22697536 (2026).
- [3] D. Faraglia, contributors, Faker: a Python package that generates fake data, <https://github.com/joke2k/faker> (2026).
- [4] B. Baudry, K. Etemadi, S. Fang, Y. Gamage, Y. Liu, Y. Liu, M. Monperrus, J. Ron, A. Silva, D. Tiwari, Generative AI to generate test data generators, IEEE Software (2024). doi:10.1109/MS.2024.3418570.
- [5] S. Kannan, Y. Chung, A. Gondi, T. Swadell, F. Özcan, High-fidelity and complex test data generation for real-world SQL code generation services, arXiv:2504.17203 (2025). doi:10.48550/arXiv.2504.17203.
- [6] L. Xu, M. Skoularidou, A. Cuesta-Infante, K. Veeramachaneni, Modeling tabular data using conditional GAN, in: Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 7335–7345.
- [7] V. Borisov, K. Seßler, T. Leemann, M. Pawelczyk, G. Kasneci, Language models are realistic tabular data generators, in: The Eleventh International Conference on Learning Representations (ICLR), 2023.
- [8] D. Chays, Y. Deng, P. G. Frankl, S. Dan, F. I. Vokolos, E. J. Weyuker, An AGENDA for testing relational database applications, Software Testing, Verification and Reliability 14 (1) (2004) 17–44. doi:10.1002/stvr.286.
- [9] A. Arasu, R. Kaushik, J. Li, Data generation using declarative constraints, in: Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, 2011, pp. 685–696. doi:10.1145/1989323.1989395.

- 653 [10] N. Bruno, S. Chaudhuri, Flexible database generators, in: Proceedings
654 of the 31st International Conference on Very Large Data Bases (VLDB),
655 2005, pp. 1097–1107.
- 656 [11] K. Houkjær, K. Torp, R. Wind, Simple and realistic data generation, in:
657 Proceedings of the 32nd International Conference on Very Large Data
658 Bases (VLDB), 2006, pp. 1243–1246.
- 659 [12] M. Emmi, R. Majumdar, K. Sen, Dynamic test input generation for
660 database applications, in: Proceedings of the 2007 International Sym-
661 posium on Software Testing and Analysis (ISSTA), 2007, pp. 151–162.
662 doi:10.1145/1273463.1273484.
- 663 [13] J. Tuya, M. J. Suárez-Cabal, C. de la Riva, Full predicate coverage
664 for testing SQL database queries, *Software Testing, Verification and*
665 *Reliability* 20 (3) (2010) 237–288. doi:10.1002/stvr.424.
- 666 [14] G. M. Kapfhammer, P. McMinn, C. J. Wright, Search-based testing of
667 relational schema integrity constraints across multiple database man-
668 agement systems, in: 2013 IEEE Sixth International Conference on
669 Software Testing, Verification and Validation (ICST), 2013, pp. 31–40.
670 doi:10.1109/ICST.2013.47.
- 671 [15] P. McMinn, C. J. Wright, G. M. Kapfhammer, The effectiveness of test
672 coverage criteria for relational database schema integrity constraints,
673 *ACM Transactions on Software Engineering and Methodology* 25 (1)
674 (2015) 1–49. doi:10.1145/2818639.
- 675 [16] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee,
676 A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, C. Raffel,
677 Extracting training data from large language models, in: 30th USENIX
678 Security Symposium, 2021, pp. 2633–2650.
- 679 [17] I. Magar, R. Schwartz, Data contamination: From memorization to ex-
680 ploitation, in: Proceedings of the 60th Annual Meeting of the Associa-
681 tion for Computational Linguistics (Volume 2: Short Papers), 2022, pp.
682 157–165. doi:10.18653/v1/2022.acl-short.18.
- 683 [18] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. L. de Lacalle,
684 E. Agirre, NLP evaluation in trouble: On the need to measure LLM

- 685 data contamination for each benchmark, in: Findings of the Association
686 for Computational Linguistics: EMNLP 2023, 2023, pp. 10776–10787.
687 doi:10.18653/v1/2023.findings-emnlp.722.
- 688 [19] S. Wang, Z. Li, H. Qian, C. Yang, Z. Wang, M. Shang, V. Kumar,
689 S. Tan, B. Ray, P. Bhatia, R. Nallapati, M. K. Ramanathan, D. Roth,
690 B. Xiang, ReCode: Robustness evaluation of code generation models,
691 in: Proceedings of the 61st Annual Meeting of the Association for Com-
692 putational Linguistics (Volume 1: Long Papers), 2023, pp. 13818–13843.
693 doi:10.18653/v1/2023.acl-long.773.
- 694 [20] M. Schäfer, S. Nadi, A. Eghbali, F. Tip, An empirical evaluation of
695 using large language models for automated unit test generation, IEEE
696 Transactions on Software Engineering 50 (1) (2024) 85–105. doi:10.1
697 109/TSE.2023.3334955.
- 698 [21] M. L. Siddiq, J. C. S. Santos, R. H. Tanvir, N. Ulfat, F. Al Rifat,
699 V. Carvalho Lopes, Using large language models to generate JUnit tests:
700 An empirical study, in: Proceedings of the 28th International Conference
701 on Evaluation and Assessment in Software Engineering (EASE), 2024.
702 doi:10.1145/3661167.3661216.
- 703 [22] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, Y. Lou,
704 Evaluating and improving ChatGPT for unit test generation, Proceed-
705 ings of the ACM on Software Engineering 1 (FSE) (2024) 1703–1726.
706 doi:10.1145/3660783.
- 707 [23] Y. Tang, Z. Liu, Z. Zhou, X. Luo, ChatGPT vs SBST: A comparative
708 assessment of unit test suite generation, IEEE Transactions on Software
709 Engineering 50 (2024) 1340–1359. doi:10.1109/TSE.2024.3382365.
- 710 [24] C. Lemieux, J. P. Inala, S. K. Lahiri, S. Sen, CodaMosa: Escaping cov-
711 erage plateaus in test generation with pre-trained large language models,
712 in: 2023 IEEE/ACM 45th International Conference on Software Engi-
713 neering (ICSE), 2023, pp. 919–931. doi:10.1109/ICSE48619.2023.0
714 0085.
- 715 [25] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, Q. Wang, Software
716 testing with large language models: Survey, landscape, and vision, IEEE

- 717 Transactions on Software Engineering 50 (4) (2024) 911–936. doi:10.1
718 109/TSE.2024.3368208.
- 719 [26] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, G. Fraser,
720 Are mutants a valid substitute for real faults in software testing?, in:
721 Proceedings of the 22nd ACM SIGSOFT International Symposium on
722 Foundations of Software Engineering (FSE), 2014, pp. 654–665. doi:
723 10.1145/2635868.2635929.
- 724 [27] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, M. Harman,
725 Mutation testing advances: An analysis and survey, in: Advances in
726 Computers, Vol. 112, Elsevier, 2019, pp. 275–378. doi:10.1016/bs.a
727 dcom.2018.03.015.
- 728 [28] G. Meszaros, xUnit Test Patterns: Refactoring Test Code, Addison-
729 Wesley, 2007.